



上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

基于卷积神经网络的图像分类任务

计算机视觉第一次作业

学院 自动化与感知学院

班级 电院 2351

学号 523031910196

姓名 王华天

2026 年 4 月 13 日

目录

1	实验背景	3
1.1	基于传统特征提取的分类	3
1.2	基于深度学习的分类	3
2	数据集说明	3
2.1	数据集基本信息	4
2.2	基于 Jittor 的数据集处理	4
2.3	数据集适用性	5
3	网络结构	5
3.1	整体网络架构	5
3.2	残差模块 (ResidualBlock) 设计	6
3.3	结构设计思路	6
4	实验过程	7
4.1	实验环境配置与初始化	7
4.2	模型实例化优化器设置	7
4.3	超参数对比实验	8
4.4	消融实验	8
5	结果分析与总结	9
5.1	超参数对比实验结果	9
5.2	消融实验结果	9
5.3	分类任务结果	10
6	实验反思	10

1 实验背景

计算机视觉注重实现机器对视觉信息的感知、理解与运用，而图像分类是计算机视觉的基础任务之一。其核心目标是将输入的图像在预设的类别标签中进行分类，是目标检测、图像分割等高级视觉任务的基础。

1.1 基于传统特征提取的分类

在深度学习技术流行前，传统的基于视觉的图像分类依赖人工设计的特征提取算法（如 SIFT[1]、HOG[2]、SURF[3] 等），需要领域专家根据图像特性手动构建特征规则。这类方法在鲁棒性上明显存在不足，对光照变化、视角偏移、图像噪声的变化敏感，泛化能力弱，难以满足复杂场景下的分类需求。

1.2 基于深度学习的分类

近年来，随着算力发展，依赖算力的深度学习技术在图像分类领域表现优异，其中卷积神经网络（CNN）凭借其能够提取局部位置特征的独特网络结构成为图像分类的主流技术。CNN 模拟生物视觉神经系统的工作机制，通过卷积层自动提取图像的边缘、纹理、轮廓等底层特征与语义特征，结合池化层实现特征降维与信息压缩，最终通过全连接层完成分类决策，实现了端到端的特征学习与模型训练。从 LeNet-5[4] 在手写数字识别任务中的成功应用，到 AlexNet[5] 引爆深度学习浪潮，再到 VGG[6]、ResNet[7]、GoogLeNet[8] 等网络结构的持续优化，CNN 已成为图像分类任务的标准方案。

与此同时，PyTorch[9]、TensorFlow[10] 等主流深度学习框架的发展，极大简化了神经网络的搭建部署流程；Jittor[11] 作为国产高性能深度学习框架，为学术研究与工程实践提供了新的选择。这些框架为深度学习算法的快速验证与落地提供了技术支撑。

本实验依托标准分类数据集 CIFAR-10，基于 Jittor 框架实现卷积神经网络的图像分类全流程，旨在掌握 CNN 的核心原理、数据预处理方法、模型设计与训练优化技巧，探究超参数与网络结构对分类性能的影响。通过实验实践加强对深度学习与计算机视觉的理解，为后续复杂视觉任务的研究与开发奠定基础。

2 数据集说明

本实验选用计算机视觉领域经典的标准图像分类数据集 CIFAR-10，通过国产框架 Jittor 内置接口完成数据集的加载、导出与预处理，采用官方标准划分版本：包含 50000 张标注训练图像、10000 张标注测试图像。

2.1 数据集基本信息

CIFAR-10 数据集由加拿大高级研究院（CIFAR）发布，是深度学习图像分类任务中最常用的小型基准数据集之一，专为验证卷积神经网络（CNN）的基础分类能力设计，广泛应用于教学实验与算法快速验证。该数据集所有图像统一为 32×32 像素的 RGB 图像，相较于 MNIST 手写数字数据集，CIFAR-10 包含真实场景物体，特征更复杂，更能检验模型的特征提取能力。

本实验采用官方标准划分方案，共计 60000 张标注图像，其中包含 50000 张标注训练图像、10000 张标注测试图像。CIFAR-10 数据集共包含 10 个互斥的物体类别，每个类别包含 6000 张图像（训练集 5000 张 + 测试集 1000 张），有效避免了数据不平衡对分类结果的干扰。10 个具体类别为：飞机、汽车、鸟类、猫、鹿、狗、青蛙、马、船、卡车。



图 1: CIFAR-10 十类别示例

2.2 基于 Jittor 的数据集处理

本实验依托 Jittor 深度学习框架完成 CIFAR-10 数据集的加载与导出，核心处理流程如下：

首先利用 Jittor 内置的 `jittor.dataset.cifar.CIFAR10` 接口，自动完成数据集的下载、加载与导出。而后为提升模型鲁棒性，在每轮训练开始时采用随机裁剪 (`transform.RandomCrop`) 与随机翻转 (`transform.RandomHorizontalFlip`) 的方式对训练集内的数据进行数据增强。具体而言，对输入的 32×32 彩色图像进行随机位置裁剪，输出尺寸仍保持 32×32 ，从而增加训练样本多样性，提升泛化能力。随机翻转同理以 50% 的概率对图像进行水平镜像翻转。

同时数据增强不会覆盖原有数据，而是在不同训练轮次 (Epoch) 中，将随机裁剪、

随机翻转生成不同的增强版本的数据作为训练数据，从而间接扩充训练样本多样性。

2.3 数据集适用性

CIFAR-10 数据集规模适中、标注规范、类别清晰，且与 Jittor 框架兼容性极佳，无需复杂的数据清洗与标注工作，非常适合用于卷积神经网络的搭建、训练、调参与性能评估，完全满足本实验的任务需求。

3 网络结构

本实验基于卷积神经网络（CNN）与残差网络（ResNet）核心思想设计图像分类模型。网络整体以残差模块为基础单元，通过分层特征提取与下采样实现高维语义特征学习，最终完成 CIFAR-10 数据集 10 分类任务。

3.1 整体网络架构

网络输入为 CIFAR-10 数据集的 3 通道 32×32 RGB 彩色图像，整体结构分为三部分：初始特征映射层、残差特征提取层、全连接分类输出层，整体网络架构如表 1 所示，核心设计如下：

1. 初始卷积层：采用 1 个 3×3 卷积核，将 3 通道输入图像映射为 64 通道特征图，完成底层图像特征的初步提取；
2. 残差阶段组：依次堆叠 4 个残差阶段，每个阶段包含 2 个 ResidualBlock 残差模块，通道数逐级递增为 64、128、256、512；其中第 1 阶段保持特征图尺寸不变，第 2、3、4 残差阶段通过步长 $\text{stride}=2$ 实现特征图下采样；
3. 分类输出层：通过自适应平均池化层（AdaptiveAvgPool2d）将高维特征压缩为固定维度，最后经全连接层输出 10 维分类结果，对应 CIFAR-10 的 10 个类别。

表 1: 网络结构核心参数表

网络模块	输入大小	步长	输出大小
输入层	-	-	$3 \times 32 \times 32$
初始卷积层 (3×3 Conv)	$3 \times 32 \times 32$	1	$64 \times 32 \times 32$
残差阶段 1 ($2 \times \text{Block}$)	$64 \times 32 \times 32$	1	$64 \times 32 \times 32$
残差阶段 2 ($2 \times \text{Block}$)	$64 \times 32 \times 32$	2	$128 \times 16 \times 16$
残差阶段 3 ($2 \times \text{Block}$)	$128 \times 16 \times 16$	2	$256 \times 8 \times 8$
残差阶段 4 ($2 \times \text{Block}$)	$256 \times 8 \times 8$	2	$512 \times 4 \times 4$
自适应平均池化 + 全连接层	$512 \times 4 \times 4$	-	10 (分类维度)

3.2 残差模块 (ResidualBlock) 设计

ResidualBlock 是本网络的核心基础单元，在卷积模块的基础之上通过添加残差模块以解决深度神经网络训练中的梯度消失问题，残差模块图示如图 2 所示，模块内部结构如下：

1. 卷积层：模块中包含两层 3×3 卷积层，用于提取图像的纹理、轮廓、语义特征；
2. 标准化层：每层卷积后依次接入批量归一化 (BatchNorm)，加速模型收敛并提升训练稳定性防止输出随训练漂移；
3. 激活层：通过较为常见的 ReLU 激活函数为网络引入非线性，使其具备通用近似能力，同时具备一定环节梯度饱和的能力；
4. 正则化：加入 Dropout 层，在训练时将一部分神经元置零，减少模型对部分神经元的依赖，有效抑制模型过拟合；
5. 残差连接：由于网络层数较深，在训练过程中容易出现梯度消失的问题，通过捷径分支使梯度可以直接回传，不被多层激活/卷积削弱；当特征图因步长变化发生下采样时，采用 1×1 卷积层对捷径分支进行维度与尺寸对齐；无维度变化时直接相加，保证残差相加的维度匹配。

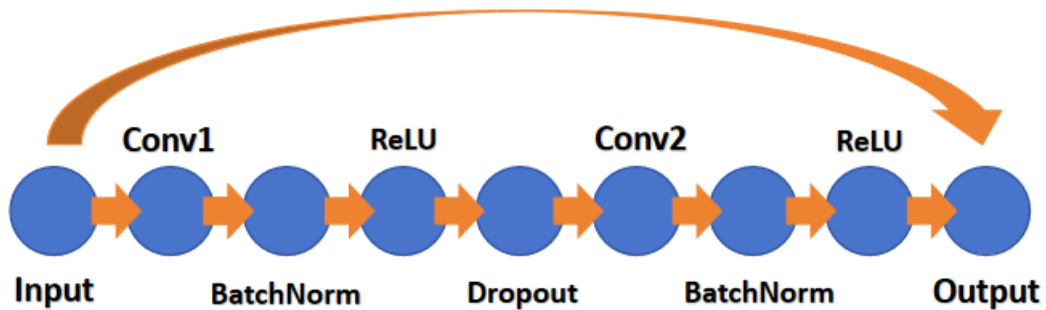


图 2: 残差模块

3.3 结构设计思路

结合 CIFAR-10 数据集的特性（小尺寸彩色图像、10 类别均衡分布），本实验中使用的网络设计思路如下。

首先在设计残差结构时采用轻量化设计，在保证模型深度的同时，避免参数量过大导致的训练缓慢或过拟合。在每个阶段逐级提升通道数配合下采样，实现从底层边缘特征到高层语义特征的分层提取，同时在每个阶段中加入 BatchNorm 与 ReLU 等经典网络设计以确保基础性能。在网络中加入残差连接保证深度网络可训练性，通过 Dropout

减少过拟合提升鲁棒性。最后的输出阶段通过自适应平均池化，无需手动计算超参数即可得到固定尺寸输出，最后通过全连接层线性变换将输出转换为 10 维，以匹配十分类任务。

4 实验过程

本实验基于 Jittor 深度学习框架完成 CIFAR-10 图像分类任务，咋在上述数据集处理以及模型网络设计的基础之上，进行了实验环境配置、模型实例化、选择合适的优化器对模型进行训练、进行超参数对比实验探讨不同超参数影响。

4.1 实验环境配置与初始化

本实验可通过脚本完成环境配置，确保环境可复现与依赖版本统一。由于 Jittor 框架与 windows 系统自带编译器不兼容，因此在实验中使用 WSL2 以确保能够基于 Linux 系统进行环境部署，通过 pyenv 管理 Python 版本，使用虚拟环境隔离依赖，并配置 CUDA、cuDNN、编译器与 Jittor 框架，避免默认配置导致版本冲突或运行异常。

表 2: 实验硬件与基础系统环境

类别	配置信息
操作系统	Ubuntu 20.04 / WSL2 (Windows 子系统)
GPU 加速卡	NVIDIA GeForce RTX 4060
显存大小	8GB
Python 版本	3.10.14
深度学习框架	Jittor
C/C++ 编译器	gcc-12、g++-12 (与 nvcc 兼容版本)
CUDA 工具链	12.2
cuDNN	对应 CUDA 版本的深度神经网络加速库

4.2 模型实例化优化器设置

实验实例化自定义 SimpleResNet 模型，设置 dropout 概率为 0.3 防止过拟合，输出类别数为 10 以匹配 CIFAR-10 分类任务。

在优化器方面，本实验采用随机梯度下降法 (SGD) 作为模型参数更新的核心优化器，并配合动量与权重衰减策略联合优化网络训练过程。相比传统梯度下降 SGD 计算量更小，收敛速度更快，且梯度带噪声更容易跳出局部最优。动量通过累积历史梯度使参数更新具备方向惯性，加速收敛并有效解决梯度震荡与局部最优等问题。

带动量的 SGD 更新公式如下：

$$v_t = \gamma \cdot v_{t-1} + \eta \cdot \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_t$$

为抑制模型过拟合，本实验引入权重衰减（L2 正则化）策略，通过在损失函数中加入参数的 L2 范数惩罚项，约束模型权重的分布范围，避免参数过大导致的过拟合现象。

权重衰减等价于给损失函数添加正则项：

$$L_{total} = L_{original} + \frac{\lambda}{2} \|\theta\|^2$$

4.3 超参数对比实验

为探究超参数对模型性能的影响，实验将模型结构固定，将动量系数与权重衰减系数固定为 0.9 与 $1e-4$ ，对学习率（LR）与批次大小（Batch Size）进行组合搜索：将学习率取值 $[0.03, 0.01, 0.003]$ ，批次大小取值 $[16, 32, 64]$ 进行共 9 组实验，在每组实验中进行 40 轮训练并对比结果。通过观察在训练过程中损失函数与测试准确率以评估性能，得到的结果如图 3 所示。

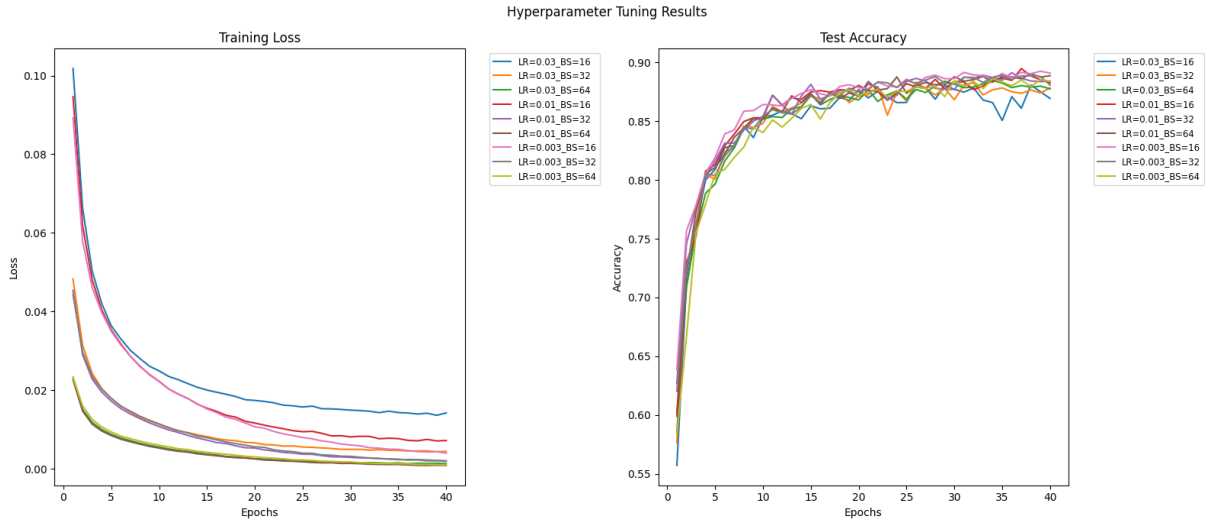


图 3: 超参数对比实验

4.4 消融实验

为验证随即裁剪、批归一化、dropout、动量与权重衰减等设计的有效性，本次探究设计了消融实验，以完整模型为基线，与上述各个模块逐个去除后的模型进行对比。对比时将学习率与批次大小固定为 0.003 与 16 进行 20 轮训练，最终通过在测试集上的准确率评估性能。得到的结果如表 3 所示。

表 3: 消融实验结果对比

配置	最终 Acc	最终 TrainLoss	最终 Acc 变化	最终 TrainLoss 变化
基线	89.75%	0.0192	+0.00pp	+0.0000
去除随机裁剪	87.89%	0.0109	-1.86pp	-0.0083
去除批归一化	85.97%	0.0254	-3.78pp	+0.0061
去除 Dropout	89.65%	0.0112	-0.10pp	-0.0080
去除动量	86.48%	0.0285	-3.27pp	+0.0093
去除权重衰减	89.21%	0.0190	-0.54pp	-0.0003

5 结果分析与总结

5.1 超参数对比实验结果

通过观察图 3 可知, 首先在批次大小的取值上, 当批次大小取值较小时, 在训练集上的损失往往会更大, 这是由于随着批次规模增大, 模型会更倾向于整个训练集的数据, 使训练损失减小。但这种拟合方式 (以及更小的训练损失) 并不代表模型能够在测试集上表现更好, 反而可能由于过度拟合训练集而缺少泛化能力。因此小批次的随机梯度下降不仅能以更快的速度进行训练同时有助于增加模型的泛化能力。

在学习率取值上, 取较高的学习率能够在训练初期以更快的速度降低损失 (如左图中蓝、红、粉线所示), 但最终会由于学习率过大面临梯度振荡等问题, 导致模型无法收敛, 也可在右图中观察到随训练轮次增大测试准确率不增反降且出现剧烈振荡, 较低学习率反而能保证训练更加稳定, 最终取得更好的效果。

综上所述最终选定学习率与批次大小分别为 0.003 与 16 以进行最终的分类任务。

5.2 消融实验结果

如表 3 所示, 在逐个去除随即裁剪、批归一化等模块后的模型在最终 Acc 的评估上相比于总模型性能均有所下降。其中随即裁剪、批归一化与动量模块对最终差异的贡献程度最大。由于数据量与模型规模均较小, 随即裁剪能大幅增加数据多样性, 提升模型泛化能力。相比之下 Dropout 虽然具备同样功能但提升幅度相对较小, 是因为模型本身较小, 更不容易出现依赖部分神经元的情况。批归一化能够优化训练过程并一定程度上降低梯度消失的可能性 (由于使用 ReLU 作为激活函数在部分不理想情况下可能出现梯度消失)。而动量模块在加速收敛的同时降低陷入局部最优的可能性。

综上所述最终选定完整模型进行最终分类任务。

5.3 分类任务结果

在上述训练参数配置确定后，在 CIFAR-10 数据集上进行 100 轮训练，得到的结果如图 4 所示。

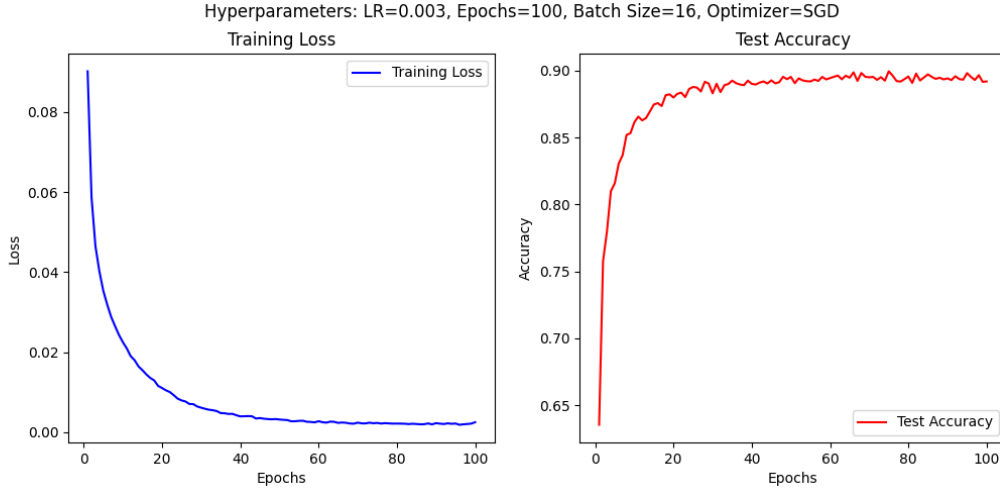


图 4: 最终分类结果

6 实验反思

由于此前没有模型设计相关经验，在模型搭建的过程中没有尝试加入特殊的设计，仅在查阅相关资料后依据经验搭建了相对较为保守同时确保能得到一定质量的结果的网络。且在训练过程中由于时间原因使用较为常见的 SGD 优化器，没有尝试一些可能提升性能的算法（如余弦退火等），仅根据数据集规模进行了数据增强的操作。后续会在进行更复杂的计算机视觉任务时尝试上述算法。

此外，我在 ai 的辅助之下完成了 Jittor 框架的部署，但在配置的过程中仍遇到相当多问题，可以说是整个流程中最大的困难，包括但不限于：windows 系统本身与 Jittor 兼容性差、在配置 WSL 时 CPU 虚拟化问题、Jittor 不支持默认环境中的 python 版本、使用 pyenv 回退版本后无法自动定位 python 配置文件只能手动生成、apt 下载的 CUDA 缺少兼容的 cudNN 等问题，最终成功配置并可在本地运行。

参考文献

- [1] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.
- [2] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, volume 1, pages 886–893. IEEE, 2005.

- [3] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. Surf: speeded up robust features. In *European Conference on Computer Vision*, pages 404–417. Springer, 2006.
- [4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, volume 25, pages 1097–1105, 2012.
- [6] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [8] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.
- [9] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*, volume 32. 2019.
- [10] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems, 2015.
- [11] Shi-Min Hu, Dun Liang, Guo-Ye Yang, Guo-Wei Yang, and Wen-Yang Zhou. Jittor: a novel deep learning framework with meta-operators and unified graph execution. *Science China Information Sciences*, 63(12):222103, 2020.
- [12] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.